

LE DEMINEUR

Rapport de la SAE21_2025

William-James et Théo TP05

SOMMAIRE

Introduction : Principe du démineur	2
Fonctionnalité du programme	3-7
Structure du Programme (Diagramme de Classe)	8
Explication du mécanisme de sauvegarde	9-10
Exposition de l'algorithme qui permet de révéler plusieurs cases	11-12
Conclusion William	13
Conclusion Théo	14

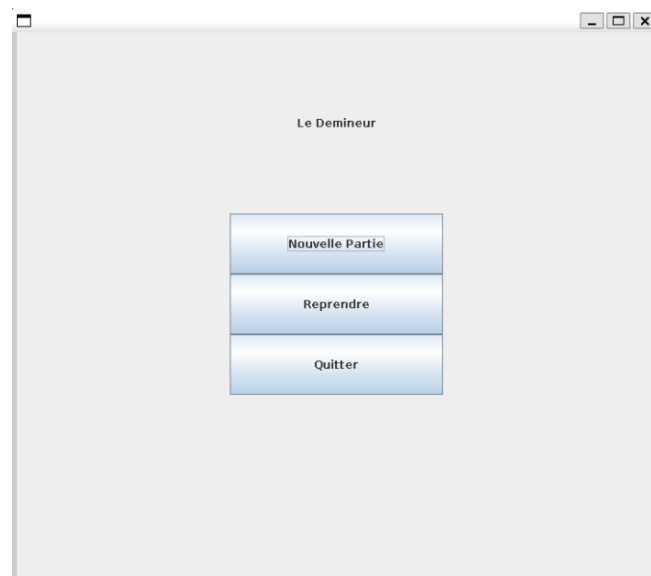
1.INTRODUCTION : LE DEMINEUR

Le programme a pour objectif de permettre à l'utilisateur de lancer une partie de démineur qui découle du jeu originel Démineur (Mine Sweeper) créé en 1990 par des employés de Microsoft en 1990 nommé **Robert Donner** et **Curt Johnson**. Ce jeu met un joueur face à une grille contenant une a plusieurs mines selon le choix du ce dernier entre plusieurs difficultés. Le but, réussir à trouver toutes les mines cachées en révélant leur case adjacente. Pour faire face à ce défi le joueur peut utiliser un système de marquage qui l'aidera dans sa réflexion, ce dernier est la plupart du temps représenté par l'utilisation de drapeau ou de point d'interrogation. Alliés à ce système de marquage le jeu mais à la disposition du joueur un algorithme de calcul qui numérote à chaque case révéler si cette dernières et adjacentes à une ou plusieurs mines. Un système de révélation récursif est également présent lors de la partie pour éviter que le joueur s'attarde sur des zones où il n'y a pas de mines a trouvé. Le joueur aura la possibilité de sauvegarder ça "grille de jeu", qui pourra ensuite relancer lors de sa prochaine connexion.

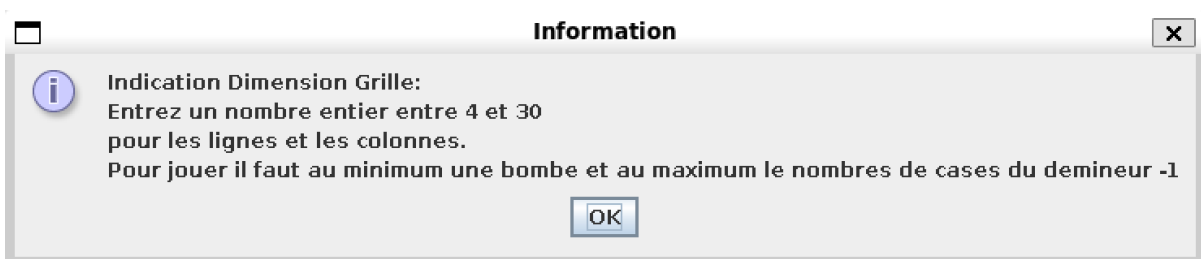
En cas de de défaite le joueur pourra voir une vue de la grille avec les cases exactes où se trouvait les mines avant de choisir de quitter l'application ou de rejouer.

2.Fonctionnalités du programme

- Menu principal : Mise à disposition de 3 possibilités pour le joueur :
 - Lancer une partie en appuyant sur “Jouer”.
 - Charger la partie que le joueur avait sauvegardé dernièrement.
 - Fermer l'application en appuyant sur “Quitter”.



Lorsque le joueur choisit l'option jouer la fenêtre de l'écran d'accueil se ferme et le joueur est redirigé vers le choix de dimension de la grille avec laquelle il jouera mais également du nombre de mines qu'il souhaite inclure dans sa partie. Mais avant d'accéder à cette page un pop-up indiquant les règles à suivre pour le remplissage (Dimension/Mines) est montré au joueur.



Ces règles seront également rappelées au joueur en cas de mauvaise saisie de sa part.

The screenshot shows a window titled "Choix dimensions" with a light gray background. On the left side, there are three labels: "LIGNES :", "COLONNES :", and "Nombre de bombe :". To the right of these labels are three empty text input fields stacked vertically. Below the input fields is a blue button with the text "VALIDER". The window has standard Windows-style window controls (minimize, maximize, close) in the top right corner.

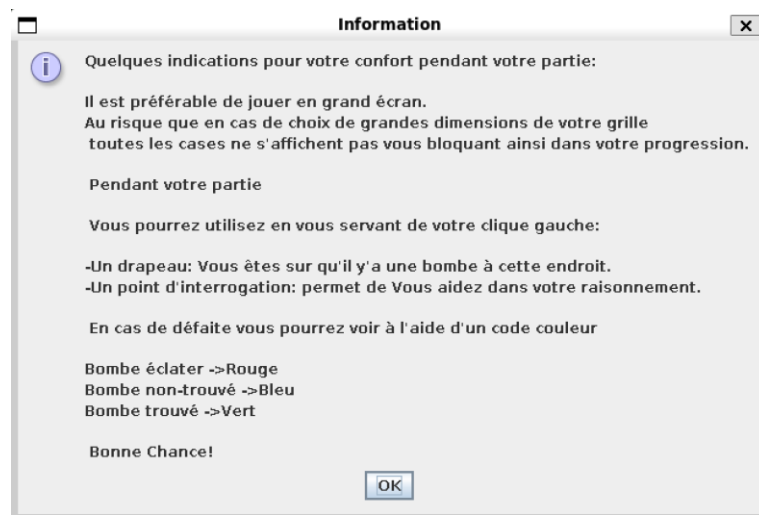
This screenshot shows the same "Choix dimensions" window, but with an error dialog box overlaid in the center. The error dialog is titled "ERREUR" and contains the following text in red: "Entrez un nombre entier entre 4 et 30 pour les lignes et les colonnes . Pour jouer il faut au minimum une bombe, et au maximum le nombre de case du demineur -1". Below the text is a blue "OK" button. The "Choix dimensions" window behind it shows the "LIGNES :" label and its corresponding input field, which appears to be empty or has a very faint value.

Les champs de textes sont également vidés automatiquement permettant ainsi à l'utilisateur de saisir directement de nouvelle valeur en accord avec les règles de personnalisation de la partie. Une fois les informations concernant la partie entré et validé le jeu commencera réellement laissant le joueur fasse à la mission de déminer le champ qui lui fait face.

Règles du jeu

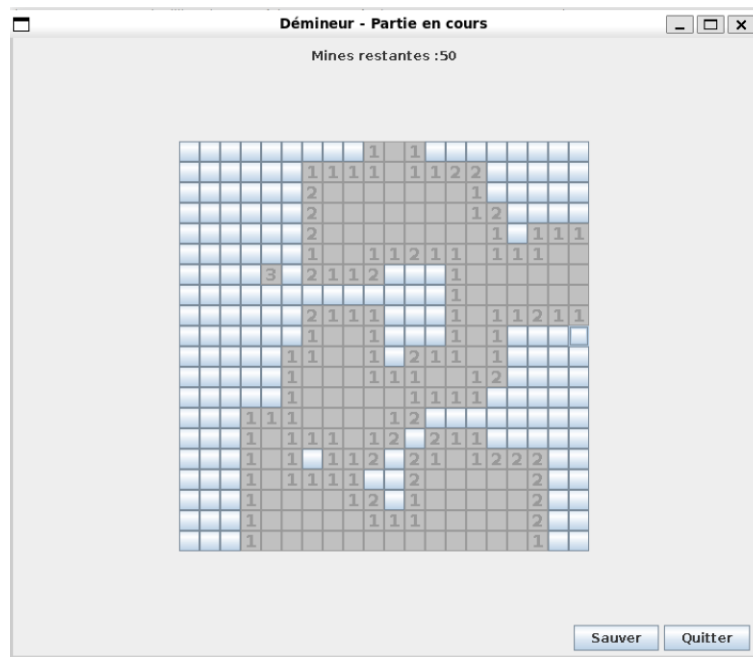
Par précaution avant de lancer le joueur dans le “grand bain” un pop-up concernant les mécaniques du jeu lui sera mis à disposition :

- Outils de réflexion (Drapeau/point d’interrogation)
- Explication brève de l’évènement de révélation récursif
- Code couleur des cases minées (en cas de défaite)
- Compteur

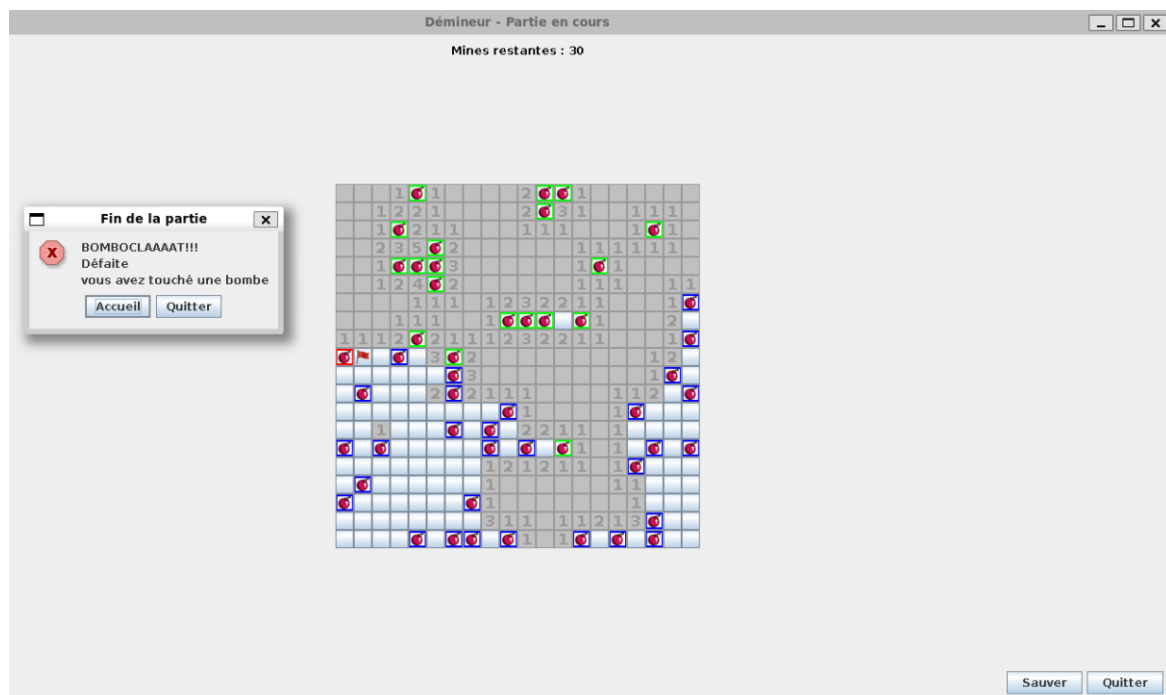


Exemple de Partie

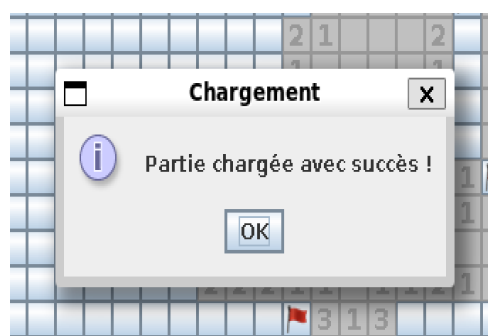
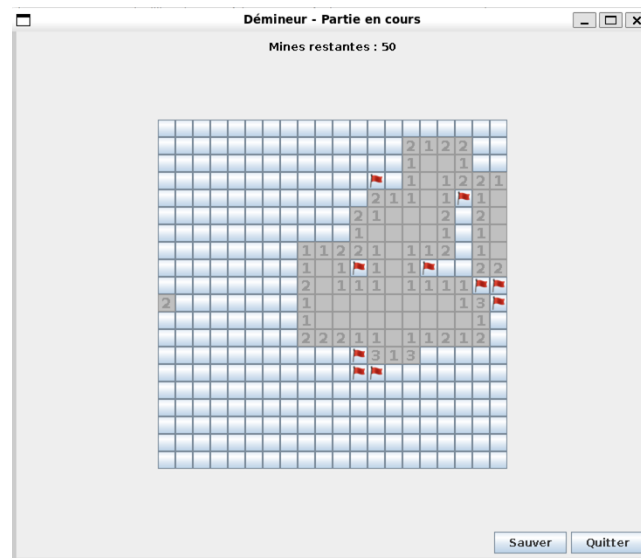
- Effet de propagation et indications des mines aux alentours



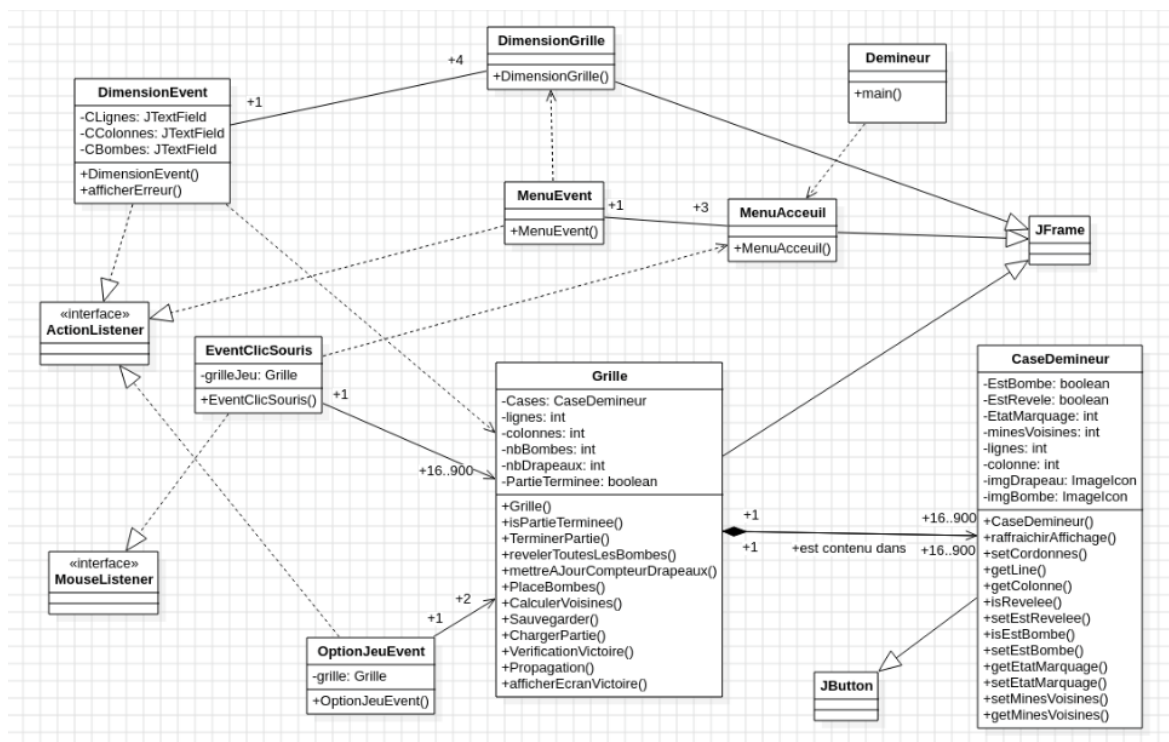
- Révélations des bombes de la partie avec le code couleur (rouge/bleu/vert)



Le joueur aura également l'occasion de sauvegarder sa partie, qu'il pourra recharger en cliquant sur le bouton "Reprendre" du menu d'accueil.



3. Structure du programme (Diagramme)



4.Explication du système de sauvegarde

Fonctionnement général

Le système de sauvegarde enregistre l'état complet d'une partie de Démineur dans un fichier texte appelé sauvegarde.txt. Cela permet au joueur de reprendre sa partie plus tard quand il le souhaite avec le bouton reprendre.

Ce qui est sauvegardé

Quand on clique sur le bouton "Sauver", le programme sauvegarde :

1. Informations globales de la partie :

- a. Les dimensions de la grille (nombre de lignes et colonnes)
- b. Le nombre total de bombes
- c. Le nombre de drapeaux déjà posés par le joueur
- d. La date et l'heure de la sauvegarde

2. L'état de chaque case :

- a. Sa position (ligne, colonne)
- b. Si elle contient une bombe ou non
- c. Si elle a été révélée ou non
- d. Son état de marquage (0 = rien, 1 = drapeau, 2 = point d'interrogation)

Format du fichier

Le fichier se présente sous cette forme :

DIMENSIONS:10,10 (grille de 10x10)

NBOMBES:4 (4 bombes au total)

DRAPEAUX_POSES:0 (0 drapeaux placés)

DATE:Thu Apr 09 21:40:29 (date et heure)

CASES: (début de la liste des cases)

0,0,false,true,0 (ligne 0, col 0, pas de bombe, révélée, pas marquée)

0,1,false,true,0 (ligne 0, col 1, pas de bombe, révélée, pas marquée)

...

Chargement de la partie

La méthode chargerPartie() lit ce fichier et :

- Reconstitue une grille identique
- Restaure l'état de chaque case (si elle est révélée, marquée, etc.)
- Met à jour le compteur de drapeaux

Ainsi, le joueur retrouve exactement sa partie telle qu'elle était avant la fermeture du jeu.

5.Exposition de l'algorithme qui permet de révéler plusieurs cases(propagation)

Fonctionnement général

Quand on clique sur une case qui ne contient pas de bombe, un algorithme se déclenche. Cet algorithme révèle automatiquement non seulement la case cliquée, mais aussi toutes les cases vides voisines adjacentes.

Fonctionnement étape par étape

La méthode propagation (int ligne, int colonne) fonctionne ainsi :

1. Vérifications initiales :

- La case doit être dans les limites de la grille (pas dehors)
- La case ne doit pas déjà être révélée (évite les boucles infinies)
- La case ne doit pas contenir une bombe

2. Révélation de la case actuelle :

- La case est marquée comme révélée

3. Décision (point clé) :

- Si la case a des mines autour (nombre de mines voisines > 0) : l'algorithme s'arrête là
- Si la case n'a pas de mines autour (nombre de mines voisines $= 0$) : l'algorithme continue

4. Propagation récursive (si aucune mine à proximité) :

- L'algorithme s'appelle lui-même pour les 8 cases voisines (haut, bas, gauche, droite, et diagonales)
 - Chaque appel répète les étapes 1 à 4
-

Exemple

On imagine cette grille (0 = pas de mines à proximité, 1+ = nombre de mines voisines, B = bombe) :

0 0 0 1 B

0 0 0 1 B

1 1 1 0 0

B B 1 0 0

Quand on clique sur le premier 0 :

1. Il se révèle
2. Comme il n'a pas de mines autour, l'algorithme propage aux 8 voisins
3. Les 0 voisins se révèlent à leur tour et propagent
4. Quand on atteint les cases avec des chiffres (1, 2...), elles se révèlent mais ne propagent pas
5. Les bombes ne sont jamais révélées

Résultat : Une grande zone vide et triangulaire de cases révélées en une seule action ce qui rend le jeu plus agréable et les parties plus courtes

Résumé

C'est un algorithme récursif assez basique ce qui simplifie le code.

5. Conclusion William

Ce projet, a été de mon point de vue révélateur au niveau de l'importance de l'organisation et de la phase de concertation entre membres du projet. J'ai pu voir à quel point le fait de se répartir chacun un rôle précis dans le développement des diverses fonctionnalités demandé permettait de gagner un gain de temps très important dans l'avancée du projet.

Nous avons pu nous projeter très vite dans les tâches que nous aurions à réaliser et au fur à mesure du temps nous avons pu nous faire des suggestions sur la mise en place de certaines méthodes et leur intégration à notre code.

Néanmoins j'ai pu également voir que la **présence de "deadline"** était également nécessaire car il peut arriver parfois de minimiser la durée de développement de certaines fonctionnalités qui au premier abord paraissent très simple et basique mais nécessitent en réalité une véritable réflexion sur le rôle que va jouer l'élément notamment avec le cas de notre **class CaseDemineur** qui nécessitait une quantité assez importante de getter et setter.

Dans l'ensemble ce projet a également été une occasion pour clarifier certain concept que je n'avais pas encore compris avant de commencer à travailler dessus notamment celui des Evènement avec les "contrôleur" de l'**interface ActionListener** a **MouseListener** avec lesquels j'ai pu avoir certaines interrogations sur la mise en place notamment des clics sur les cases pour placés élément de réflexion du joueur (drapeau, point d'interrogations) tels que "comment la souris sait si j'ai fait un clique droits ou un clic gauche?" moments où la **Javadoc** a été plutôt utile.

J'ai également pu apprendre qu'on pouvait intégrer du **html** et du **css** dans notre code par l'intermédiaire de certains camarades ce qui fut très surprenant car je pensais que ce système de "communication" entre langage était spécifique aux langages web.

Pour conclure je dirais que c'est surtout l'aspect **organisation et communication** régulière entre membre de projets qui a été au centre de ce travail.

4. Conclusion Théo

Avec William, on a créé un jeu de Démineur complet qui fonctionne vraiment. J'ai réussi à faire trois grandes parties : la sauvegarde du jeu, l'algorithme qui révèle les cases vides automatiquement, et les écrans de victoire et défaite.

Ce projet m'a permis d'apprendre à bien organiser mon code. J'ai séparé mon travail dans plusieurs classes différentes, chacune avec son rôle spécifique notamment avec une roadmap pour que l'on puisse se séparer les tâches. J'ai compris comment fonctionnent les algorithmes difficiles, notamment la propagation qui utilise la récursion. J'ai aussi vu que faire un vrai programme, c'est compliqué. Il faut gérer l'interface, les clics de la souris, la logique du jeu et la sauvegarde en même temps.

En codant ce projet, j'ai découvert que la récursion, c'est plus facile qu'on le pense quand on comprend bien le problème. J'ai aussi appris que sauvegarder et charger une partie, ça demande vraiment du travail. Il faut lire le fichier ligne par ligne, vérifier que tout est correct, et tout reconstruire. Les détails qu'on oublie, comme vérifier si une case est dans la grille, peuvent casser tout le jeu.

Ce projet montre que j'ai compris les bases de la programmation et que j'arrive à faire quelque chose qui marche vraiment. Le jeu est jouable, le code est bien optimisé, et il n'y a pas d'erreur lors de la compilation ou lors du test du jeu.

FIN